# QMWS Fall 2026: Introduction to Python for Data Analysis

## Day Three: Applications

Gavin Klorfine & Deborah Laze

Section 1

# matplotlib.pyplot

# matplotlib.pyplot

- **matplotlib.pyplot** is a package for visualizing data in Python
- Fun fact: If you have asked ChatGPT for a graph, you likely received one that was made using this package

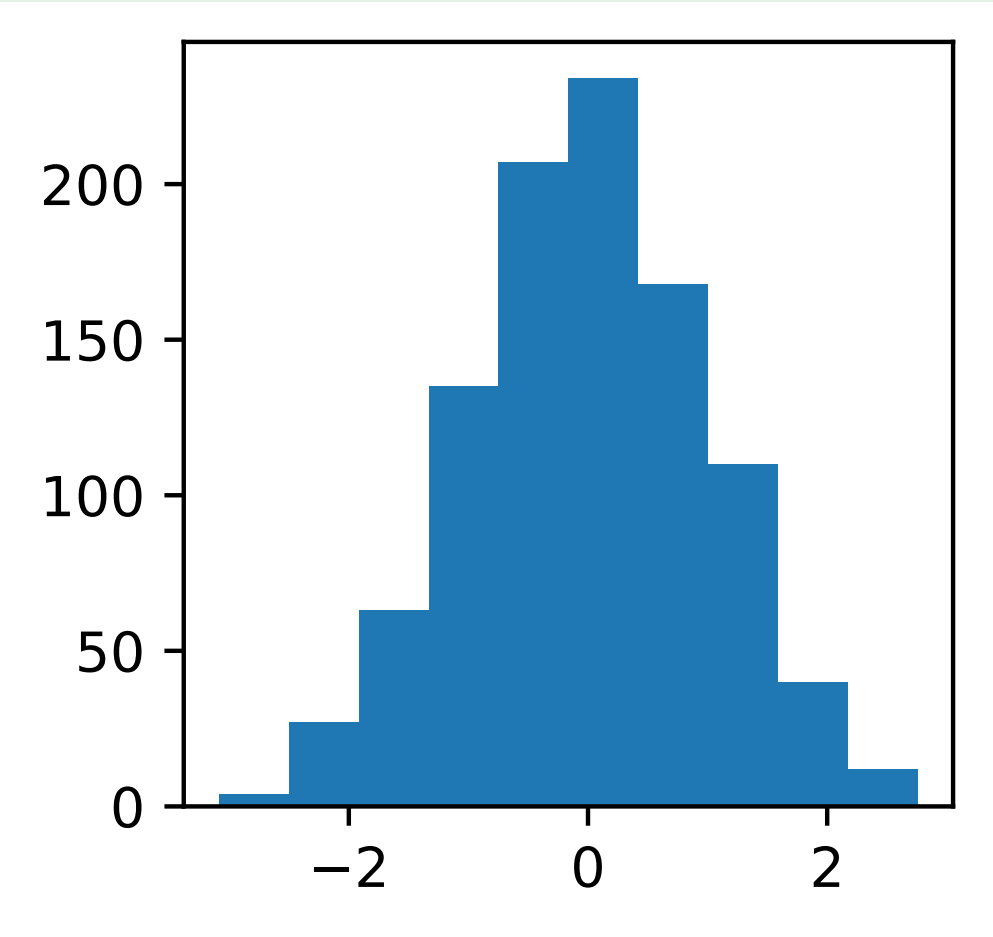
# matplotlib.pyplot

## Basic Histogram

```
import numpy as np
import matplotlib.pyplot as plt

x = np.random.normal(0, 1, 1000)
plt.hist(x)
plt.show()
```

## Basic Histogram

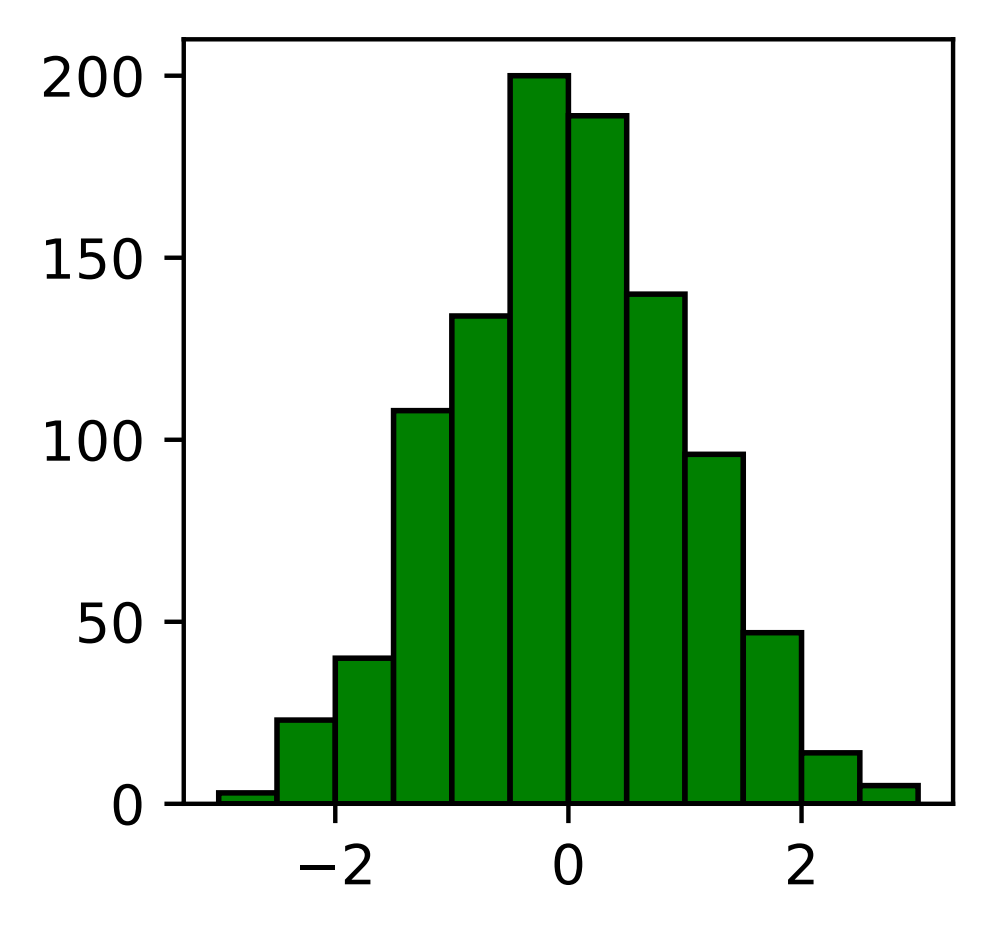


- A few ways to modify the histogram are illustrated below

## Modifying the Plot

```
plt.hist(
    x,                          # Data
    color = "green",            # Bar Colour
    edgecolor = "black",        # Outline Colour
    bins = np.arange(-3, 3.5, 0.5)#Bins of 0.5 from -3 to +3
)
plt.show()
```

## Modifying the Plot

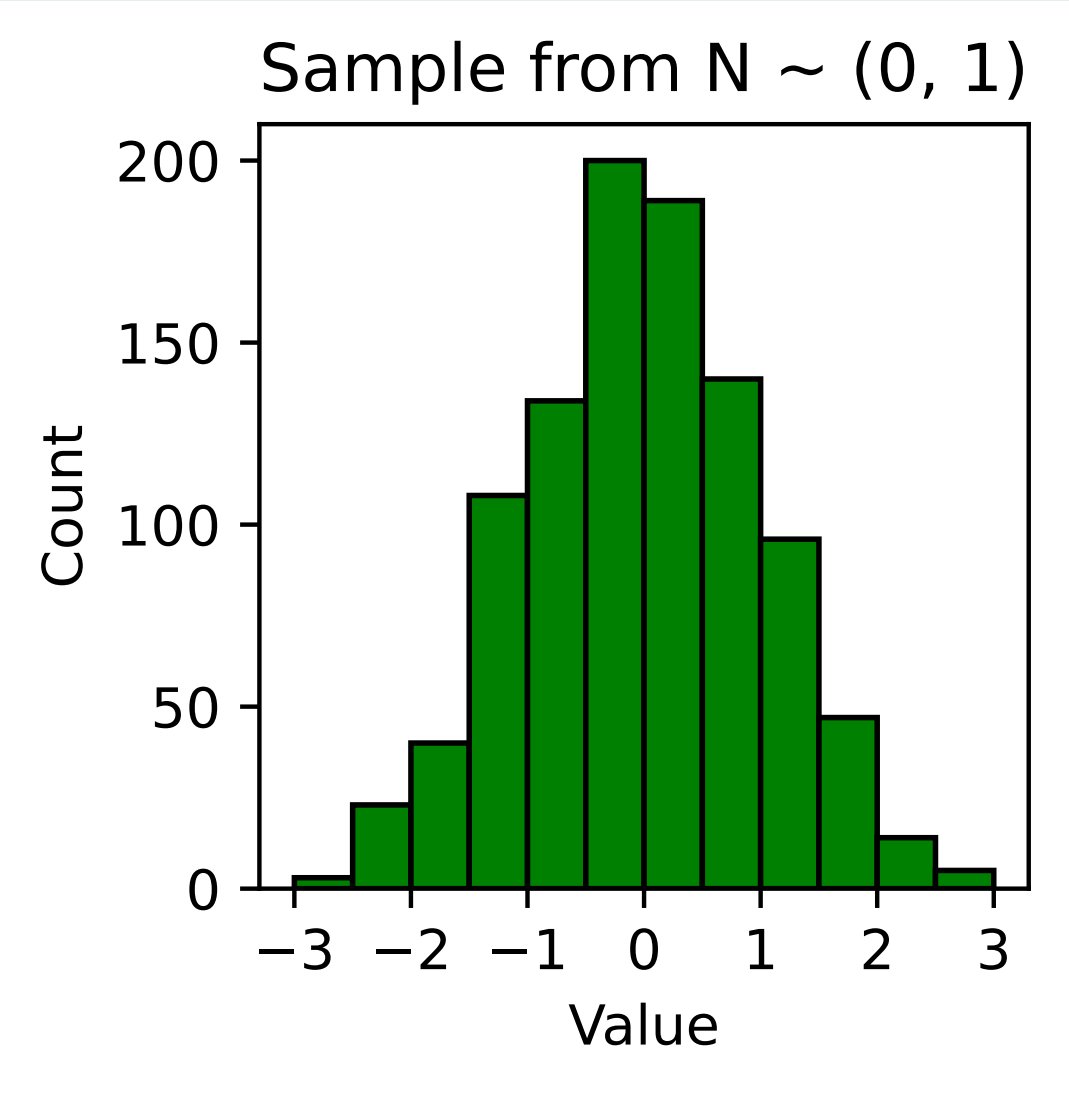


## More Ways to Modify

```
plt.hist(
    x, color = "green", edgecolor = "black",
    bins = np.arange(-3, 3.5, 0.5)
)

plt.xticks(np.arange(-3, 4, 1)) # x-axis ticks spaced by 1
plt.title("Sample from N ~ (0, 1)")
plt.xlabel("Value")
plt.ylabel("Count")
plt.show()
```

## More Ways to Modify



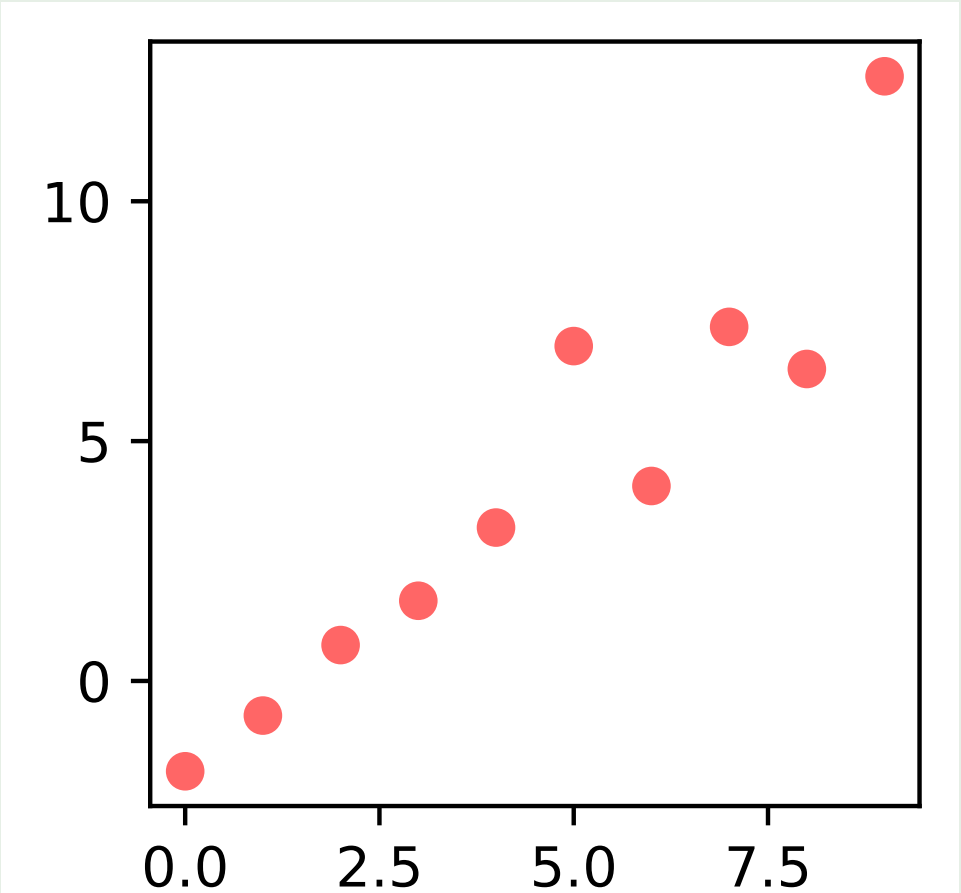
- The below is how data was generated for subsequent scatterplot examples

## Simulated Data for a Simple Regression

```
errors = np.random.normal(0, 2, 10) # Normal errors
nums_x = np.arange(0, 10, 1) # Nums 0 to 9
nums_y = nums_x + errors # y = x + errors
```

## Scatterplot

```
# plot(x, y, style, transparency)
plt.plot(nums_x, nums_y, 'or', alpha=0.6)
plt.show()
```



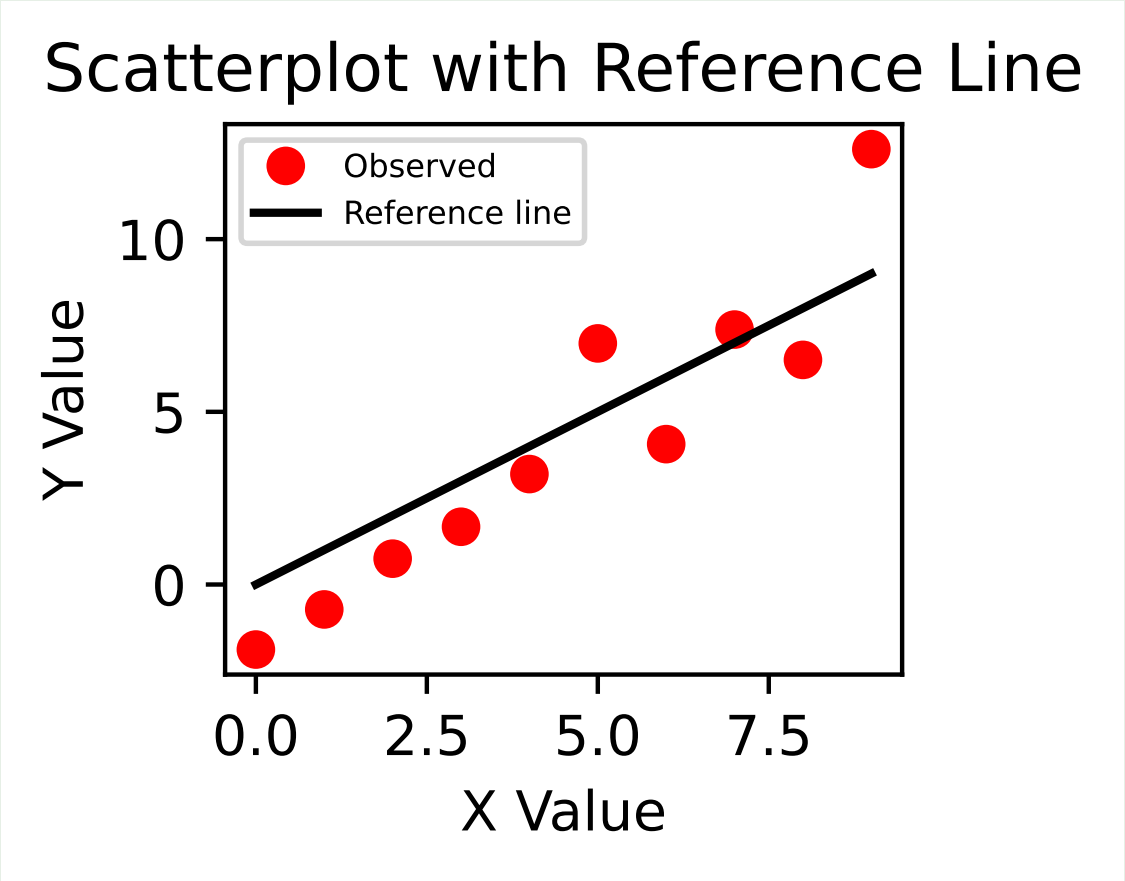
# matplotlib.pyplot

- You can add another plot layer by calling another plotting function before `plt.show()`
- This is useful for adding:
  - reference lines
  - fitted lines
  - multiple groups

## Adding a Line to a Scatterplot

```
#Adds observed data as red circles
plt.plot(nums_x, nums_y, 'or')
#Adds reference line (x=y) as black solid line
plt.plot(nums_x, nums_x, '-k')
plt.show()
```

## Adding Labels and a Legend




- `label` gives each layer a name
- `plt.legend()` displays the labels on the plot

## Adding Labels and a Legend, continued

- `plt.title()`, `plt.xlabel()`, and `plt.ylabel()` make the plot easier to interpret

Section 2

# statsmodels

# statsmodels

- **statsmodels** is a package for statistical testing and modelling
- It is useful when you want:
  - formal hypothesis tests
  - regression models
  - $p$-values and confidence intervals
  - detailed statistical output

## Importing **statsmodels** Functions

```
import statsmodels.formula.api as smf
from statsmodels.stats.weightstats import ttest_ind
```

# statsmodels

- **statsmodels.formula.api** contains functions that use formula notation
- It is commonly renamed `smf`
- `ttest_ind` is a function for independent-samples $t$-tests

## Import Structure

```
import statsmodels.formula.api as smf
#      package.module          alias

from statsmodels.stats.weightstats import ttest_ind
#    package.module.submodule        function
```

## Statistical Testing

- Statistical tests help us use a **sample** to make inferences about a larger population
- A test usually starts with a **null hypothesis**
  - Often "no difference" or "no relationship"
- We compare the observed data to what would be expected if the null hypothesis were true

**Note**

- The code runs the test, but the researcher chooses whether the test makes sense

## General Workflow

1. State the research question
2. Identify the level of measurements and study design
3. Summarize and visualize the data
4. Run the appropriate test
5. Interpret the estimate, confidence interval, and $p$-value

# statsmodels

## Example Data

- We will use simulated scores from two independent groups
- Each row represents one participant
- The group labels are stored in a pandas DataFrame

## Creating the Data

```
import pandas as pd

np.random.seed(42)

df = pd.DataFrame({
    "group": ["control"] * 30 + ["treatment"] * 30,
    "score": np.concatenate([
        np.random.normal(70, 10, 30),
        np.random.normal(76, 10, 30)
    ])
})
```

## Inspecting the Data

```
print(df.head())    # First five rows
print(df.shape)     # (number of rows, number of columns)
```

```
     group      score
0  control  74.967142
1  control  68.617357
2  control  76.476885
3  control  85.230299
4  control  67.658466
(60, 2)
```

- Always inspect the data before running a test

## Independent-Samples $t$-test

- Used to compare the means of **two independent groups**
- Do not use this test if the same participants appear in both groups
- Two versions exist depending on whether the groups' variances are assumed equal — both use `ttest_ind()`, just with a different `usevar` argument
- We'll work through this test using our general workflow

**Research Question:**

- Do treatment and control groups have different mean scores?

**Null hypothesis:**

$$H_0 : \mu_{\text{treatment}} - \mu_{\text{control}} = 0$$

## Independent-Samples $t$-test: Level of Measurement

- `group`: categorical (nominal), two independent levels — grouping variable
  - Study design: independent (between-subjects) groups
- `score`: continuous (interval/ratio) — outcome variable

## Independent-Samples $t$-test: Inspecting the Data

```
print(df.groupby("group")["score"].describe().round(2))
```

| group | count | mean | std | min | 25% | 50% | 75% | max |
|---|---|---|---|---|---|---|---|---|
| control | 30.0 | 68.12 | 9.00 | 50.87 | 64.09 | 67.66 | 73.60 | 85.79 |
| treatment | 30.0 | 74.79 | 9.31 | 56.40 | 68.91 | 75.35 | 81.45 | 94.52 |

- Always re-check the data immediately before running a test
- `groupby("group")` summarizes the control and treatment rows separately
- `.describe()` gives common summary statistics for each group

## Selecting Each Group

```
control = df.loc[
    df["group"] == "control",  # Rows to include
    "score"                    # Column to return
]

treatment = df.loc[
    df["group"] == "treatment",
    "score"
]
```

- `ttest_ind()` needs the two groups as separate variables

## Independent-Samples $t$-test: The `usevar` Argument

```
ttest_ind(
    x1,                  # First group's values
    x2,                  # Second group's values
    alternative = "two-sided",
    usevar = "pooled",   # or "unequal" (Welch's)
    value = 0            # Null-hypothesis difference
)
```

- `usevar = "pooled"` assumes equal population variances (the regular independent-samples $t$-test)
- `usevar = "unequal"` does **not** assume equal population variances (**Welch's $t$-test**) — a useful default when in doubt

## Independent-Samples $t$-test: The `usevar` Argument

- Use `alternative = "two-sided"` when either group could have the larger mean
- Use `value = 0` when the null hypothesis is no mean difference

## Independent-Samples $t$-test: Running the Test

```
from statsmodels.stats.weightstats import ttest_ind

pool_result = ttest_ind(
    treatment, control, usevar = "pooled"
)
welch_result = ttest_ind(
    treatment, control, usevar = "unequal"
)
```

- Only the `usevar` argument changes between the two tests

## Independent-Samples $t$-test: Results

- `ttest_ind()` returns three values: the $t$-statistic, the $p$-value, and the degrees of freedom.

```
print(pool_result)
print(welch_result)
```

```
(np.float64(2.8210747685177l), np.float64(0.006542565964576836
(np.float64(2.8210747685177l), np.float64(0.006544738502602257
```

```
# You can also index these results:
print(f"{pool_result[0]},"
f"{pool_result[1]}, {pool_result[2]}")
print(f"{welch_result[0]},"
f"{welch_result[1]}, {welch_result[2]}")
```

```
2.8210747685177l,0.006542565964576836, 58.0
2.8210747685177l,0.006544738502602257, 57.93320998252982
```

## $t$-Test Interpretation

- The $t$-statistic describes the group difference relative to its uncertainty
- The $p$-value is the probability, assuming the null hypothesis is true, of observing a test statistic at least as extreme as the one observed
- The degrees of freedom help determine the $t$-distribution used by the test

**Important**

- A $p$-value is **not** the probability that the null hypothesis is true

## Calculating the Mean Difference

```
mean_difference = treatment.mean() - control.mean()

print(f"Treatment mean: {treatment.mean():.2f}")
print(f"Control mean: {control.mean():.2f}")
print(f"Mean difference: {mean_difference:.2f}")
```

```
Treatment mean: 74.79
Control mean: 68.12
Mean difference: 6.67
```

- The $t$-test output should be interpreted alongside the observed group difference
- The mean difference itself does not depend on the variance assumption, so it is the same for both versions

## Interpreting the Results: Equal Variances

```
print(
    f"Mean difference = {mean_difference:.2f}\n"
    f"t({pool_result[2]:.2f}) = {pool_result[0]:.2f}\n"
    f"p = {pool_result[1]:.3f}"
)
```

```
Mean difference = 6.67
t(58.00) = 2.82
p = 0.007
```

- Assumes the two groups have equal population variances

## Interpreting the Results: Unequal Variances (Welch's)

```
print(
    f"Mean difference = {mean_difference:.2f}\n"
    f"t({welch_result[2]:.2f}) = {welch_result[0]:.2f}\n"
    f"p = {welch_result[1]:.3f}"
)
```

```
Mean difference = 6.67
t(57.93) = 2.82
p = 0.007
```

- Does **not** assume equal population variances
- Compare the two reports above — here the conclusion is the same either way, which is common when group variances are similar

## Correlation

- A correlation describes the direction and strength of a **linear relationship**
- Pearson's correlation coefficient is represented by $r$
- It ranges from -1 to 1

**Research Question:**

- Is there a linear relationship between study hours and score?

**!** Important

- Correlation does not establish causation

## Correlation: Level of Measurement

- `study_hours`: continuous (interval/ratio)
- `score`: continuous (interval/ratio)
- Study design: observational/correlational (no groups being compared)
- Both variables are continuous, so a Pearson correlation is appropriate

## Adding a Second Variable

```
np.random.seed(7)

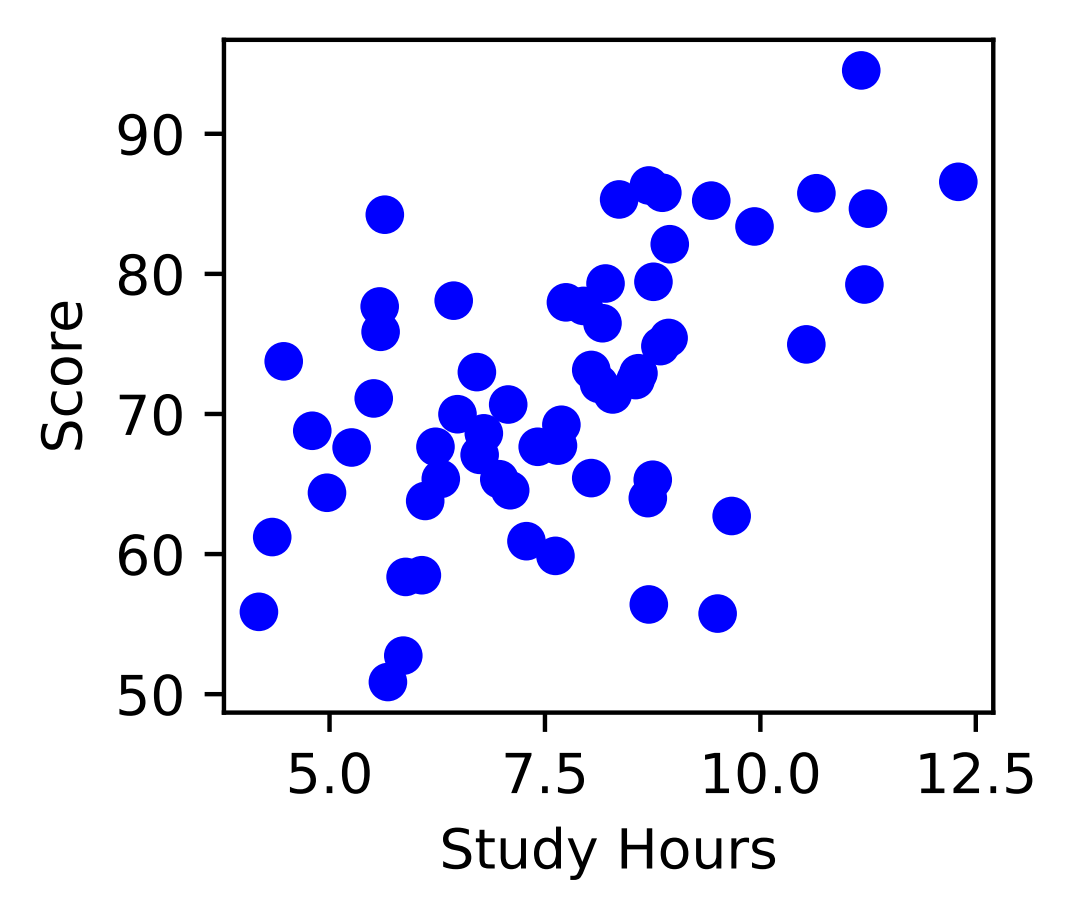
df["study_hours"] = (
    2
    + 0.08 * df["score"]
    + np.random.normal(0, 1.5, len(df))
)
```

## Scatterplot

```
plt.plot(df["study_hours"], df["score"], "ob")
plt.xlabel("Study Hours")
plt.ylabel("Score")
plt.show()
```

- Use a scatterplot before calculating a correlation — this is the *inspecting the data* step of our workflow

## Scatterplot



## .corr() Method

```
correlation = df["study_hours"].corr(
    df["score"]
)

print(f"r = {correlation:.2f}")
```

```
r = 0.53
```

- .corr() describes the observed sample relationship

## Correlation: Running the Test

- `.corr()` gives the **estimate** ($r$) of the linear relationship, but not a $p$-value or confidence interval on its own
- In **simple linear regression** with one predictor, the slope's significance test is mathematically equivalent to testing whether $r$ differs from zero
- We'll get the confidence interval and $p$-value for this same relationship next, using regression

## Simple Linear Regression

- Simple linear regression examines the relationship between:
  - one continuous **outcome**
  - one continuous **predictor**
- Here, the outcome is `score` and the predictor is `study_hours`

**Research Question (same as above):**

- Is there a linear relationship between study hours and score?

$$\text{score} = \text{intercept} + (\text{slope} \times \text{study hours}) + \text{error}$$

## Simple Linear Regression: Level of Measurement

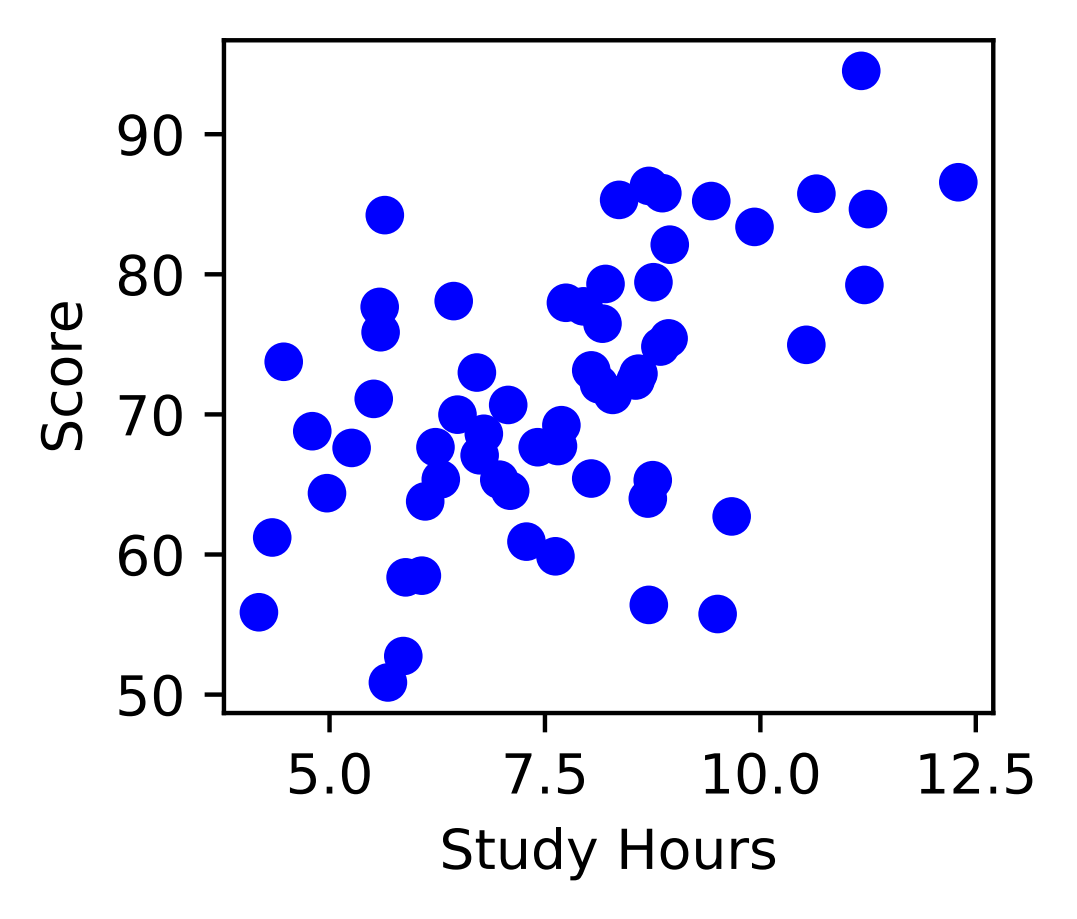
- `study_hours`: continuous (interval/ratio) — predictor
- `score`: continuous (interval/ratio) — outcome
- Study design: observational/correlational, same as above

## Simple Linear Regression: Inspecting the Data

```
plt.plot(df["study_hours"], df["score"], "ob")
plt.xlabel("Study Hours")
plt.ylabel("Score")
plt.show()
```

- Same scatterplot as before — always inspect the data before fitting a model

## Simple Linear Regression: Inspecting the Data



## Formula Notation: Formula Structure

```
"score ~ study_hours"
# outcome ~ predictor
```

- The outcome goes on the left of ~
- The predictor goes on the right of ~
- Column names must match the DataFrame

# statsmodels

## Creating a Model with `smf.ols()`

```
model_definition = smf.ols(
    formula = "score ~ study_hours",
    data = df
)

print(type(model_definition))
```

```
<class 'statsmodels.regression.linear_model.OLS'>
```

- `ols` stands for **ordinary least squares**
- `smf.ols()` defines the model but does not estimate its coefficients yet

## Fitting a Model with `.fit()`

```
model = model_definition.fit()

print(type(model))
```

- `.fit()` estimates the intercept, slope, and other model statistics
- Store the fitted result so its attributes can be inspected

## Method Chaining: Defining and Fitting in One Step

```
model = smf.ols(
    "score ~ study_hours",
    data = df
).fit()
```

- This is a common way to define and fit a model

## Model Parameters with `.params`

```
print("Intercept:", round(model.params["Intercept"], 2))
print("study_hours:", round(model.params["study_hours"], 2))
```

```
Intercept: 50.43
study_hours: 2.75
```

- `Intercept`: predicted score when study hours equals zero
- `study_hours`: predicted score change for one additional study hour

## Accessing One Parameter

```
intercept = model.params["Intercept"]
slope = model.params["study_hours"]

print(f"Intercept: {intercept:.2f}")
print(f"Slope: {slope:.2f}")
```

```
Intercept: 50.43
Slope: 2.75
```

- Use the parameter name to extract a specific estimate

## Testing the Slope with .pvalues

```
slope_p_value = model.pvalues["study_hours"]

print(f"study_hours p-value: {slope_p_value:.3f}")
```

```
study_hours p-value: 0.000
```

**Null hypothesis:**

$$H_0 : \text{population slope} = 0$$

## Confidence Intervals with `.conf_int()`

```
slope_ci = model.conf_int().loc["study_hours"]

print(f"Lower: {slope_ci[0]:.2f}")
print(f"Upper: {slope_ci[1]:.2f}")
```

```
Lower: 1.59
Upper: 3.90
```

- A confidence interval gives a range of values compatible with the model and data

## Model Overview: `.summary().tables[0]`

```
summary = model.summary()
print(summary.tables[0])
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  score   R-squared:                       0.280
Model:                            OLS   Adj. R-squared:                  0.268
Method:                 Least Squares   F-statistic:                     22.57
Date:                Sat, 31 Oct 2026   Prob (F-statistic):           1.37e-05
Time:                        14:28:52   Log-Likelihood:                -210.99
No. Observations:                  60   AIC:                             426.0
Df Residuals:                      58   BIC:                             430.2
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
```

- Table 0 summarizes the model setup and overall fit
- Useful for checking the outcome, sample size, $R^2$, and model type

## Model Overview: `.summary().tables[1]`

```
print(summary.tables[1])
```

```
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      50.4278      4.553     11.076      0.000      41.314      59.541
study_hours     2.7466      0.578      4.751      0.000       1.589       3.904
==============================================================================
```

- Table 1 gives the estimates for each model term
- Focus first on the coefficient, $p$-value, and confidence interval columns

## Model Overview: `.summary().tables[2]`

```
print(summary.tables[2])
```

```
==============================================================================
Omnibus:                        0.940   Durbin-Watson:                   1.864
Prob(Omnibus):                  0.625   Jarque-Bera (JB):                0.843
Skew:                          -0.281   Prob(JB):                        0.656
Kurtosis:                       2.852   Cond. No.                         34.0
==============================================================================
```

- Table 2 contains diagnostic and assumption-related information
- Useful later when checking whether the model is behaving reasonably

## .rsquared

- $R^2$ is the proportion of outcome variance described by the fitted model
- It ranges from 0 to 1
- A larger $R^2$ does not prove that the model is correct or causal

```
print(f"R^2: {model.rsquared:.3f}")
```

```
R^2: 0.280
```

## Predicted Outcomes with .predict()

```
df["predicted_score"] = model.predict(df)

for i in range(3):
    print(
        f"Observed: {df['score'][i]:.2f}; "
        f"Predicted: {df['predicted_score'][i]:.2f}"
    )
```

```
Observed: 74.97; Predicted: 79.36
Observed: 68.62; Predicted: 69.08
Observed: 76.48; Predicted: 72.86
```

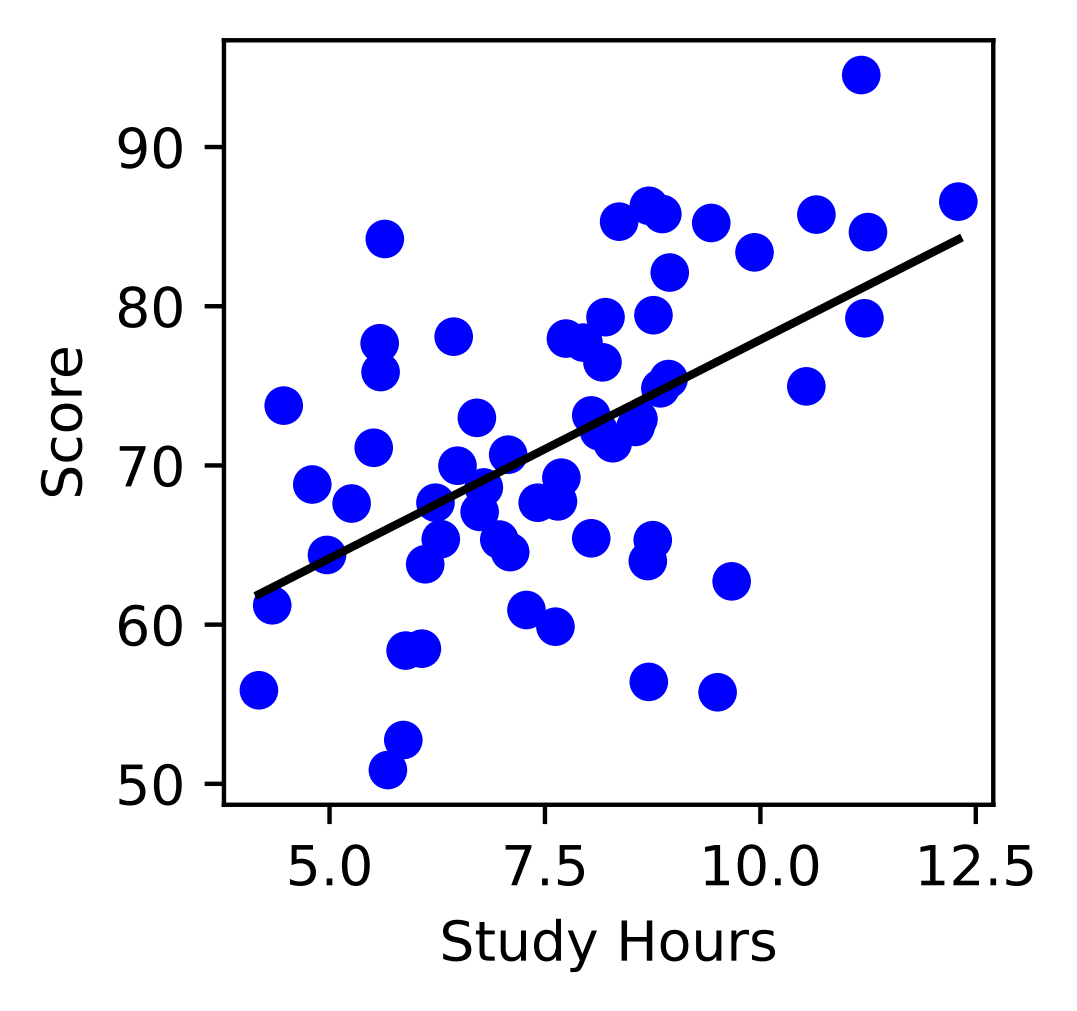
- .predict() calculates the fitted outcome for each row

## Plotting Model Predictions: Observed and Predicted Values

```
ordered = df.sort_values("study_hours")

plt.plot(df["study_hours"], df["score"], "ob")
plt.plot(
    ordered["study_hours"], ordered["predicted_score"], "-k"
)
plt.xlabel("Study Hours")
plt.ylabel("Score")
plt.show()
```

## Fitted Line




- Blue circles are observed values
- The black line contains model-predicted values

## Reporting Regression Results: Formatting the Result

```
slope_ci = model.conf_int().loc["study_hours"]

print(f"b = {model.params['study_hours']:.2f}, "
    f"95% CI [{slope_ci[0]:.2f}, {slope_ci[1]:.2f}], "
    f"p = {model.pvalues['study_hours']:.3f}")
```

```
b = 2.75, 95% CI [1.59, 3.90], p = 0.000
```

- Report the estimated slope, confidence interval, and $p$-value together
- Use **associated with**, rather than **caused**
- This matches the significance test for the Pearson correlation computed earlier — the slope's $p$-value is mathematically identical to the correlation's $p$-value

## Multiple Regression

- Multiple regression extends simple regression to **more than one predictor**
- Predictors can be continuous or categorical — categorical predictors are automatically dummy-coded in formula notation
- Here we add `group` as a second predictor alongside `study_hours`

**Research Question:**

- Does study hours predict score, after accounting for group — and does the effect of study hours depend on group?

$$\text{score} = \text{intercept} + b_1(\text{study hours}) + b_2(\text{group}) + \text{error}$$

## Multiple Regression: Level of Measurement

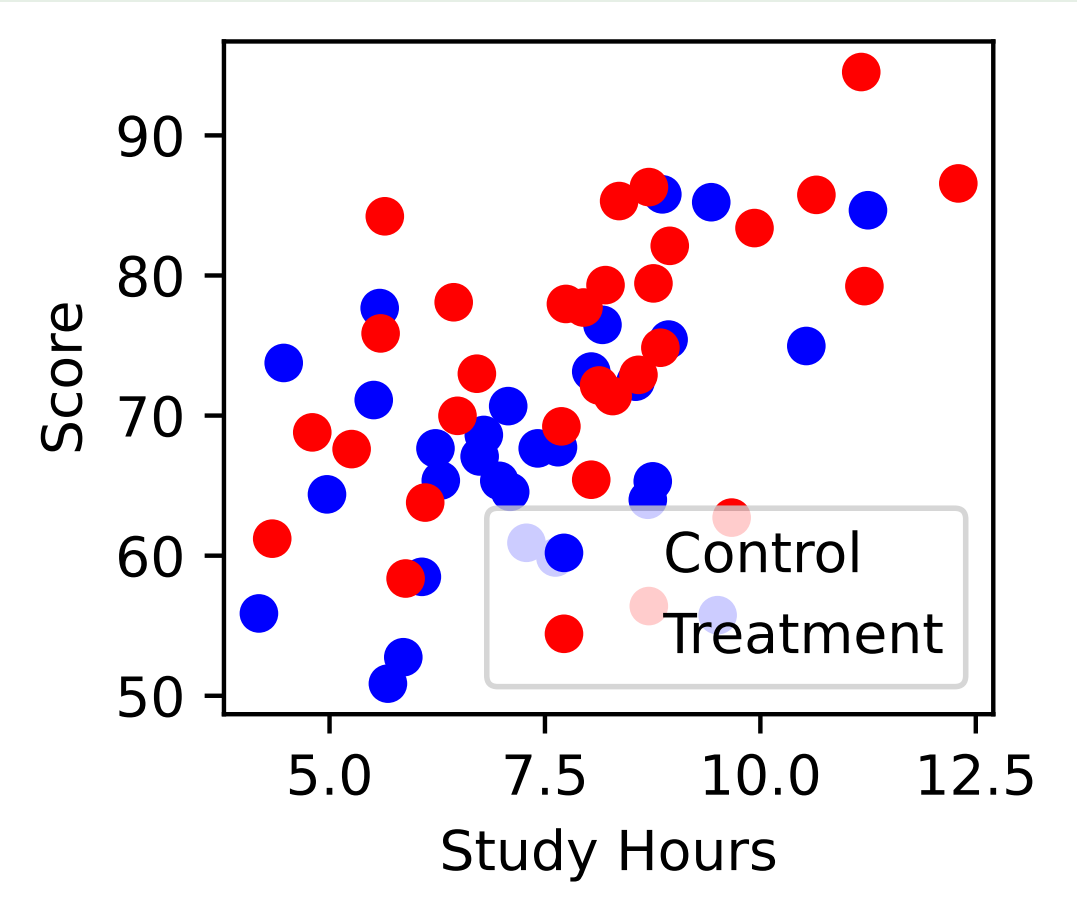
- `study_hours`: continuous (interval/ratio) — predictor
- `group`: categorical (nominal), two levels — predictor (dummy-coded)
- `score`: continuous (interval/ratio) — outcome

## Multiple Regression: Inspecting the Data

```
control_rows = df[df["group"] == "control"]
treatment_rows = df[df["group"] == "treatment"]

plt.plot(
    control_rows["study_hours"], control_rows["score"],
    "bo", label = "Control")
plt.plot(
    treatment_rows["study_hours"], treatment_rows["score"],
    "ro", label = "Treatment")
plt.xlabel("Study Hours")
plt.ylabel("Score")
plt.legend()
plt.show()
```

## Multiple Regression: Inspecting the Data



## Multiple Regression: Formula Notation

```
"score ~ study_hours + group"
# Main effects only

"score ~ study_hours * group"
# Main effects AND their interaction; equivalent to:
"score ~ study_hours + group + study_hours:group"
```

- `+` includes each predictor's **main effect** only, holding the other predictor constant
- `*` includes the main effects **and** the interaction between them

## Multiple Regression: Running the Test

```
main_effects_model = smf.ols(
    "score ~ study_hours + group", data = df
).fit()

interaction_model = smf.ols(
    "score ~ study_hours * group", data = df
).fit()
```

- Only the formula string changes between the two models

## Multiple Regression: Interpreting the Results (Main Effects)

```
main_effects_model.summary()
```

```
Intercept: b = 49.68, 95% CI [40.91, 58.45], p = 0.000
group[T.treatment]: b = 5.09, 95% CI [0.91, 9.26], p = 0.018
study_hours: b = 2.51, 95% CI [1.38, 3.64], p = 0.000
```

## Multiple Regression: Interpreting the Results: (Main Effects)

- Each coefficient is interpreted *holding the other predictor constant*

## Multiple Regression: Interpreting the Results (Interaction Effects)

```
interaction_model.summary()
```

```
Intercept: b = 49.73, 95% CI [36.69, 62.77], p = 0.000
group[T.treatment]: b = 4.99, 95% CI [-
13.00, 22.98], p = 0.581
study_hours: b = 2.51, 95% CI [0.77, 4.24], p = 0.005
study_hours:group[T.treatment]: b = 0.01, 95% CI [-
2.29, 2.31], p = 0.991
```

## Multiple Regression: Interpreting the Results (Interaction Effects)

- Switching + to * added exactly one new term: `study_hours:group[T.treatment]`
- It tests whether the `study_hours` slope differs between `control` and `treatment`
- Here it is far from significant and barely shifts the other coefficients — evidence the effect of `study_hours` does **not** depend on `group`

## Comparing the Two Models: Running the Test

```
from statsmodels.stats.anova import anova_lm

comparison_table = anova_lm(
    main_effects_model, interaction_model
)
print(comparison_table)
```

```
   df_resid           ssr  df_diff  ss_diff         F    Pr(>F)
0      57.0  3604.470329      0.0      NaN       NaN       NaN
1      56.0  3604.462629      1.0   0.0077   0.00012  0.991312
```

## Comparing the Two Models: Interpreting the Results

- Tests whether adding the interaction explains significantly more variance in `score` than the main-effects-only model
- Use this $p$-value, not the individual interaction term's $p$-value, to judge whether the interaction belongs in the model
- Here, $p$ is far from significant — the interaction does not improve the model